Mavzu: SQL tilida jadvallar va so‘rovlarni yaratish

Ma’lumotlar bazasining jadvallari massiv ko‘rinishida bo‘lib, ular satr va ustunlardan iborat bo‘ladi. Jadval tuzilgandan keyin uni to‘ldirishga kirishish mumkin. Jadvallar *CREATE TABLE* komandasi bilan yaratiladi. Bu komanda qatorlarsiz bo‘sh jadval yaratadi. *CREATE TABLE* komandasi jadval nomini va jadvalning o‘zini ma’lum tartibda ko‘rsatilgan ustunlar nomlari ketma-ketligi ta’rifi ko‘rinishida aniqlaydi. U ma’lumotlar tiplari va ustunlar o‘lchovini aniqlaydi. Har bir jadval juda bo‘lmaganda bitta ustunga ega bo‘lishi kerak. Har bir ustun uchun tip (toifa) va o‘lcham aniqlanadi. Xar bir yaratilgan jadval hech bo‘lmaganda bitta ustunga ega bo‘dishi kerak *CREATE TABLE* komandasi sintaksisi quyidagicha:

CREATE TABLE jadval_nomi (

Ustun1 ma’lumot_turi [cheklovlar],

Ustun2 ma’lumot_turi [cheklovlar],

...

);

Jadval yaratishning asosiy sintaksisi quyidagicha:

- jadval_nomi**:** Yaratilayotgan jadvalning nomi.

- ustun: Jadval ustunining nomi.

-ma’lumot_turi: Ustun uchun ma’lumot turi *(masalan, INT,VARCHAR, DATE).*

- **cheklovlar:** Ustun uchun cheklovlar *(PRIMARY KEY, NOT NULL va boshqalar)*

Oddiy jadval yaratish

Talabalar ma’lumotlarini saqlash uchun oddiy jadval yaratish:

CREATE TABLE talabalar (

ID INT PRIMERY KEY

Ism VARCHAR (50) NOT NULL,

Familiya VARCHAR (50)

Tug‘ulgan sana DATE

);

Tavsif:

- ID**:** Butun sonli ustun, asosiy kalit sifatida ishlatiladi.
- Ism**:** 50 ta belgidan oshmaydigan matn, bo‘sh qiymatlar qabul qilinmaydi.
- Familiya**:** 50 ta belgidan oshmaydigan matn, bo‘sh qiymatlar qabul qilinishi mumkin.
- Tugilgan_sana**:** Sana formati.

Cheklovlar (constraints) ma’lumotlarning yaxlitligini ta’minlash uchun ishlatiladi.

1-jadval.

Cheklovlar turlari

Cheklov turi	**Tavsifi**	**Misol**
NOT NULL	Ustunda *NULL* qiymatlar bo‘lishini taqiqlaydi.	*name VARCHAR(50) NOT NULL*
UNIQUE	Ustundagi barcha qiymatlar yagona (takrorlanmas) bo‘lishi kerak.	*email VARCHAR(100) UNIQUE*
PRIMARY KEY	Jadvallar uchun yagona identifikator bo‘lib, *NOT NULL* va *UNIQUE* kombinatsiyasidan iborat.	*id INT PRIMARY KEY*
FOREIGN KEY	Bir jadvaldagi ustunni boshqa jadvaldagi *PRIMARY KEY* bilan bog‘laydi.	*student_id INT REFERENCES students(id)*

CHECK	Ustundagi qiymatlar muayyan shartni bajarishi kerak.	*age INT CHECK (age >= 18)*
DEFAULT	Agar kiritilmasa, ustun uchun standart *(default)* qiymatni belgilaydi.	*status VARCHAR(20) DEFAULT "active"*
INDEX	Jadvalda qidiruv va saralash jarayonlarini tezlashtirish uchun indeks yaratadi.	*CREATE INDEX idx_name ON students(name)*

Cheklovlarni qo'shish

- ***NOT NULL*** – Maydon bo'sh bo'lishi mumkin emas:

```
CREATE TABLE Talaba (
        ID  INT   PRIMARY KEY,
        Ism VARCHAR (50) NOT NULL
);
```

- ***DEFAULT*** – Maydon uchun standart qiymat belgilash:

```
CREATE TABLE Talabalar  (
        ID  INT   PRIMARY KEY,
        Yosh  INT  DEFAULT 18
);
```

- ***CHECK*** – Shart qo'yish:

```
CREATE TABLE Talabalar  (
        ID  INT   PRIMARY KEY,
        Yosh  INT  CHECK  (Yosh  > =  18)
);
```

Bir nechta jadvallar bilan ishlash

Bog'langan jadvallar yaratish

"Talabalar" va "Guruhlar" jadvallari o'rtasida bog'lanish yaratish uchun

chet kalit (*FOREIGN KEY)* ishlatiladi.

REATE TABLE Guruhlar (

ID INT PRIMARY KEY,

Nomi VARCHAR(50) NOT NULL

);

CREATE TABLE Talabalar (

ID INT PRIMARY KEY,

Nomi VARCHAR(50) NOT NULL,

Guruh ID INT,

OREIGN KEY (Guruh id) REFERENCES Guruhlar (ID)

);

isol: Cheklovlar bilan jadval yaratish

CREATE TABLE XODIMLAR (

D INT PRIMERY KEY

Ism VARCHAR (50) NOT NULL,

Maosh DECIMAL(10, 2) CHECK (Maosh > 0),

Lavozim VARCHAR (50) DEFAULT 'Xodim'

);

Tavsif:

- ID: Asosiy kalit, noyob va bo'sh qiymatlar qabul qilmaydi.

- Maosh: Musbat qiymat bo'lishi kerak (CHECK cheklovi).

- Lavozim: Agar qiymat kiritilmasa, standart qiymat "Xodim" bo'ladi (*DEFAULT)*

Jadvalni o'zgartirish

Agar mavjud jadvalga yangi ustun qo'shish yoki boshqa o'zgartirishlar kiritish kerak bo'lsa, *ALTER TABLE* buyrug'i ishlatiladi.

Misollar:

1. Yangi ustun qo'shish:

ALTER TABLE Talabalar ADD Tug'ulgan sana DATE;

2. Ustunni o'zgartirish:

ALTER TABLE Talabalar MODIFY Ism VARCHAR (100);

3. Ustunni o‘chirish;

ALTER TABLE Talabalar DROP COLUMN Tug‘ulgan sana;

Bu ma’lumot SQL tilida jadval yaratish, o‘zgartirish va cheklovlar bilan ishlashni o‘rganishga yordam beradi

SQL– tilida so‘rovlar

SQL tilida so‘rovlar *(Queries)* — bu ma’lumotlar bazasidan kerakli ma’lumotlarni aniqlangan shartlar asosida tanlab olish, saralash va tahlil qilish jarayonidir. So‘rovlar orqali foydalanuvchi jadvaldagi ma’lumotlarni ko‘rishi, ularni filtrlay olishi, tartiblashi va hatto hisob kitoblar ham amalga oshirishi mumkin. So‘rovlar yordamida foydalanuvchi bir nechta jadvaldagi ma’lumotlarni bog‘lab ko‘rishi, umumiy natijalarni tahlil qilishi, shartli operatorlar orqali tanlovlar yaratishi ham mumkin. Shu sababli SQL so‘rovlar ma’lumotlar bazasining yuragi hisoblanadi, chunki ular foydalanuvchining ma’lumotlarga tez va samarali kirishini ta’minlaydi.

So‘rovlar ikki turga bo‘linadi: oddiy (asosiy) va murakkab (kompleks) so‘rovlar. Oddiy so‘rovlar bitta jadvaldan ma’lumotlarni oladi, murakkab so‘rovlar esa bir nechta jadvallar orasidagi aloqalardan foydalanadi. Masalan, *JOIN* operatorlari bir nechta jadvaldagi ma’lumotlarni birlashtirish uchun, *GROUP BY* va *HAVING* operatorlari esa statistik tahlillar va guruhlash amallarini bajarish uchun ishlatiladi. Bundan tashqari, *WHERE* sharti yordamida kerakli yozuvlarni aniqlash, *ORDER BY* bilan natijalarni tartiblash va *DISTINCT* yordamida takroriy qiymatlarni chiqarib tashlash mumkin. Yuqoridagi SQL tilidagi asosiy so‘rovlar qisqacha ifodalangan. Endilikda ular mazmunan tahlil qilinib, sintaksis jihatdan izohlanadi hamda turli misollar yordamida amalda qo‘llanilishi ko‘rsatiladi.

SQL so‘rovlarning asosini *SELECT* operatori tashkil etadi. U jadvaldan ma’lumot tanlab olish uchun ishlatiladi. Masalan:

*SELECT * FROM Talabalar;*

Bu yerda *SELECT* — ma’lumotni tanlash buyrug‘i, * — barcha ustunlarni bildiradi, *FROM* esa ma’lumot olinadigan jadval nomini ko‘rsatadi. Ushbu so‘rov

bajarilganda tizim “Talabalar” jadvalidagi barcha yozuvlarni chiqaradi. Biroq real loyihalarda barcha ustunlarni chaqirish tavsiya etilmaydi, chunki bu ortiqcha yuklama bo‘lishi mumkin. Aniq ustunlarni ko‘rsatish samaraliroq:

SELECT FIO, Kurs FROM Talabalar;

Bu so‘rov bajarilganda tizim faqat ikki ustunni chiqaradi va ortiqcha ma’lumot yuklanmaydi. Demak, *SELECT* operatori ma’lumotni aniq va maqsadli tanlash imkonini beradi.

WHERE operatori va filtratsiya jarayoni

Ko‘pincha foydalanuvchiga barcha yozuvlar emas, balki ma’lum shartga mos keladigan ma’lumot kerak bo‘ladi. Buning uchun *WHERE* operatori ishlatiladi. Masalan:

SELECT FIO, Kurs
FROM Talabalar
WHERE Kurs = 2;

Bu so‘rov bajarilganda tizim jadvaldagi barcha yozuvlarni tekshiradi va kurs qiymati 2 ga teng bo‘lgan yozuvlarni chiqaradi. Bu jarayon filtratsiya deyiladi. Bundan tashqari, murakkab shartlar ham berish mumkin:

WHERE Kurs = 2 AND FIO LIKE 'A%';

Bu yerda *AND* operatori ikki shartni birlashtiradi, *LIKE* esa matnli qidiruvni amalga oshiradi. % belgisi istalgan belgilar ketma-ketligini bildiradi. Natijada faqat ismi “A” bilan boshlanadigan 2-kurs talabalar chiqariladi.

ORDER BY orqali natijani tartiblash

So‘rov natijasini ma’lum tartibda ko‘rish ko‘pincha muhim hisoblanadi. Buning uchun ORDER BY operatori qo‘llaniladi:

SELECT FIO, Kurs
FROM Talabalar
ORDER BY FIO ASC;

Bu yerda *ASC* — o‘sish tartibini bildiradi. Agar *DESC* yozilsa, kamayish tartibida saralanadi. So‘rov bajarilganda tizim natijalarni alfavit bo‘yicha tartiblab beradi. Shuningdek tartiblash bir nechta ustun bo‘yicha ham amalga oshiriladi:

ORDER BY Kurs ASC, FIO DESC;

Bu yerda avval Kurs bo‘yicha, so‘ng FIO bo‘yicha tartiblash amalga oshadi.

Agregat funksiyalar va hisoblash jarayoni

SQL so‘rovlar faqat ma’lumot chiqarish emas, balki hisoblash imkonini ham beradi. Buning uchun agregat funksiyalar ishlatiladi:

SELECT COUNT(*) FROM Talabalar;

Bu so‘rov jadvaldagi yozuvlar sonini hisoblaydi.

SELECT AVG(Baho) FROM Baholar;

Bu esa o‘rtacha bahoni hisoblaydi.

SUM() yig‘indini, *MIN()* eng kichik qiymatni, *MAX()* eng kattasini topadi. Ushbu funksiyalar bajarilganda tizim barcha mos yozuvlarni hisoblab, yagona natija qaytaradi.

GROUP BY orqali guruhlash

Agregat funksiyalar ko‘pincha *GROUP BY* bilan birga ishlatiladi:

SELECT Kurs, COUNT(*) AS TalabalarSoni

FROM Talabalar

GROUP BY Kurs;

Bu so‘rov bajarilganda tizim talabalarni Kurs bo‘yicha guruhlarga ajratadi va har bir guruhdagi yozuvlar sonini hisoblaydi. Agar guruhlangan natijaga shart qo‘yish kerak bo‘lsa, *HAVING* ishlatiladi:

HAVING COUNT(*) > 5;

Bu 5 tadan ortiq talaba bor kurslarni chiqaradi.

JOIN operatori va jadvallarni bog‘lash

Ma’lumotlar ko‘pincha bir nechta jadvalda saqlanadi. Ularni birlashtirish uchun *JOIN* ishlatiladi:

SELECT Talabalar.FIO, Fanlar.FanNomi

FROM Talabalar

INNER JOIN TalabaFan

ON Talabalar.TalabaID = TalabaFan.TalabaID

INNER JOIN Fanlar

ON TalabaFan.FanID = Fanlar.FanID;

Bu so‘rov bajarilganda tizim uchta jadvalni bog‘lab, talabaning qaysi fan o‘qiyotganini chiqaradi. *INNER JOIN* faqat mos yozuvlarni chiqaradi. *LEFT JOIN* esa chap jadvaldagi barcha yozuvlarni saqlaydi.

Subquery (ichki so‘rov) ishlash prinsipi

Subquery — bu boshqa so‘rov ichida joylashgan so‘rovdir:

SELECT FIO

FROM Talabalar

WHERE TalabaID IN (

SELECT TalabaID

FROM Baholar

WHERE Baho = 5

);

Ichki so‘rov avval bajariladi va 5 baho olgan TalabaID larni qaytaradi. So‘ng tashqi so‘rov shu ID lar asosida talabalarni chiqaradi. Bu murakkab tahliliy vazifalarda juda foydali.

Ma’lumotni o‘zgartiruvchi so‘rovlar

SQL yordamida ma’lumotni qo‘shish, yangilash va o‘chirish mumkin.

INSERT INTO Talabalar (FIO, Kurs)

VALUES ('Aliyev Bekzod', 1);

Bu yangi yozuv qo‘shadi.

UPDATE Talabalar

SET Kurs = 3

WHERE TalabaID = 1;

Bu mavjud yozuvni o‘zgartiradi.

DELETE FROM Talabalar

WHERE TalabaID = 5;

Bu yozuvni o‘chiradi. Har doim *WHERE* bilan ishlash tavsiya etiladi.

Shuningdek, SQL tilida so‘rov yozish ma’lumotlarni boshqarishning eng muhim bosqichlaridan biri hisoblanadi. Hozirgi kunda kelib SQL so‘rovlarining amaliy ahamiyati quyidagilardan iborat:

- SQL so‘rovlarining oddiy sintaksisi va keng imkoniyatlari uni turli sohalarda (bank, tibbiyot, ta’lim, e-tijorat, IT va boshqalar) qo‘llash uchun universal vositaga aylantiradi;
- SQL orqali ma’lumotlarning ishonchliligi, samaradorligi va boshqaruv osonligini ta’minlash mumkin.

SQL tili orqali so‘rovlar yozish ma’lumotlar bazasini boshqarishda hal qiluvchi ahamiyatga ega. Turli funktsiyalar va operatorlar yordamida murakkab ma’lumotlarni boshqarish va tahlil qilishni osonlashtirishga imkon beradi. Har bir so‘rov o‘z vazifasiga ko‘ra optimallashtirilishi va samarali ishlatilishi lozim. Shu sababli, SQL tili so‘rovlarini to‘g‘ri tuzish va ularning imkoniyatlarini chuqur o‘rganish hozirgi kunda dolzarb muammolardan biri hisoblanadi.

Nazorat savollari:

1. SQL tilida jadval yaratish qanday amlga oshiriladi ?
2. Cheklovlarni qo‘shish qanday amalga oshiriladi ?
3. Cheklovlar turlari qaysilar ?
4. Jadvalni yaratish jarayonida qo‘llaniladigan ma’lumot turlari nimalardan iborat ?
5. SQL tilidagi so‘rovlar qanday amlga oshiriladi ?
6. *SELECT* so‘rovi qanday amlga oshiriladi ?
7. *INSERT* so‘rovi qanday amlga oshiriladi ?
8. *UPDATE* so‘rovi qanday amlga oshiriladi ?
9. *DELETE* so‘rovi qanday amlga oshiriladi ?
10. *WHERE* shart operatori nima ?